

CrossAudit: A Git-Native, Cross-Vendor Audit Loop for Agentic Science

Zhaohe Dong

Independent Researcher

dongzhaohe321418@gmail.com

July 2026

Abstract

Autonomous “AI scientist” systems increasingly generate, execute, and review research. In current frontier systems, however, the reviewing agent is typically drawn from the same model family or vendor as the generating agent, exposing supervision to self-preference bias and to blind spots shared across a common training pipeline; the supervision trace, moreover, usually lives in opaque platform logs rather than in an artefact a third party can replay. We present **CrossAudit**, a lightweight, infrastructure-minimal protocol for supervising autonomous research pipelines, built on three commitments: (i) *heterogeneity* — every experiment increment produced by a generator agent is audited by an agent from a different model vendor, against a versioned, human-authored rulebook (the *Constitution*); (ii) a *git-native ledger* — experiments, audit reports, verdicts, disputes and escalations are all commits, so the entire supervision history is replayable, diffable and citable; and (iii) *graded, escalation-based human oversight* — deterministic, model-free checks and hard inconsistencies gate the pipeline, judgement calls do not, and a bounded revision loop escalates unresolved blockers to a human principal. We specify the protocol as six invariants, describe a reference implementation built entirely from GitHub Actions and a few hundred lines of dependency-light Python, and report a live deployment in a computational-chemistry pipeline whose generator (Claude-based), auditor (GPT-lineage), and compute (cloud HPC) are mutually decoupled. We argue that cross-vendor audit trails of this kind offer a practical, immediately adoptable substrate for accountable machine science.

1 Introduction

Two developments frame this paper. The first is the arrival of genuinely agentic research systems: pipelines in which large language model (LLM) agents propose hypotheses, write and execute code, run experiments, and draft manuscripts with limited human intervention. Sakana’s AI Scientist produced machine-generated papers end-to-end [1, 2] and has since seen machine-led work reach peer-reviewed venues [3]; DeepMind’s AI co-scientist organises hypothesis generation as a multi-agent tournament [4]; autonomous laboratories close the loop through to physical synthesis [7, 8]. Publishers and funders are already being urged to respond institutionally [11].

The second development is older and less glamorous: science’s continuing struggle with reliability. A majority of surveyed researchers have failed to reproduce a published result [10]. Agentic pipelines raise the stakes on both sides of this ledger. They can enforce discipline no human sustains — every increment logged, every parameter recorded — or they can mass-produce plausible, well-formatted, wrong results faster than any human audit culture can absorb.

The critical question is therefore not whether machine-generated science should be supervised, but *by what mechanism*, and *by whom*. Current frontier systems embed supervision *inside* the system: a reflection agent, an automated reviewer, a ranking tournament [1, 4]. These internal critics share a structural weakness: critic and creator are instantiated from the same model or, at best, the same vendor’s training pipeline. LLM evaluators are known to systematically favour outputs resembling their own generations — self-preference bias, with self-recognition capability causally implicated [5];

judge-position and style biases compound the problem [6]. Two agents that share a training distribution share blind spots, and a reviewer that shares the author’s blind spots approves precisely the author’s characteristic errors. Meanwhile, the supervision trace of most agentic systems — who flagged what, what was revised, what was waved through — lives in ephemeral context windows or proprietary platform logs, invisible to the communities that will be asked to trust the resulting science.

Contributions. This paper makes four contributions.

1. We articulate the *same-source supervision problem* in agentic science: internal critique by same-vendor models is structurally exposed to correlated failure (§2).
2. We specify **CrossAudit**, a supervision protocol defined by six invariants — vendor heterogeneity, ledger completeness, citation validity, determinism-first, bounded revision, and graded interruption — that together yield a replayable, third-party-auditable supervision history (§3).
3. We describe a reference implementation requiring no infrastructure beyond two git repositories, a CI service, and two model API keys, and its live deployment in the author’s computational-chemistry pipeline (§4).
4. We give an explicit threat model, including the attacks the protocol does *not* withstand, and situate the design against plausible alternatives (§5–§6).

CrossAudit is deliberately modest in its claims. It does not certify scientific truth; no textual audit can. It certifies something narrower and checkable: that every increment of a machine-generated research record passed, or failed, a named set of versioned rules, applied by an agent with no stake in the increment’s acceptance, with the whole exchange preserved verbatim in a public ledger. We contend that this narrower property is the right first target for accountable machine science, for the same reason continuous integration — not formal verification — became the load-bearing quality mechanism of modern software.

2 Related work and the same-source problem

Agentic science systems. The AI Scientist [1, 2, 3] automates the ideation–experiment–writeup–review cycle, including an internal automated reviewer scored against human reviewer data. The AI co-scientist [4] structures generation–reflection–ranking–evolution agents into a self-improving tournament over hypotheses, with reported expert-validated outcomes in drug repurposing and antimicrobial resistance. Language agents for literature synthesis reach or exceed expert performance on retrieval-grounded tasks [9]. In the laboratory, Coscientist executed chemistry protocols from natural-language goals [7] and A-Lab ran autonomous materials synthesis campaigns [8]. Across all of these, quality control is architecturally *internal*: reviewer agents, reflection stages, or tournament judges instantiated from the same model family as the generators they judge.

Why internal critique is not enough. Panickssery et al. demonstrate that LLM evaluators recognise their own generations and rate them preferentially, with fine-tuning experiments showing self-recognition capability *causes* the inflation [5]. Zheng et al. catalogue further judge pathologies — position, verbosity, and self-enhancement biases — even in strong models [6]. These results concern single-model self-evaluation, but the mechanism generalises: models sharing a training pipeline share stylistic priors and knowledge gaps, so a same-vendor critic sits at the wrong end of the bias distribution precisely when it matters. We call this the *same-source supervision problem*. Cross-vendor pairing does not eliminate correlated error — all frontier models ingest substantially the same public literature, so where the literature is wrong, both can be confidently wrong together — but it removes the sharpest known bias (self-preference) and decorrelates vendor-idiosyncratic failure modes. What cross-vendor pairing cannot fix, a model-free layer must (§3, **I4**).

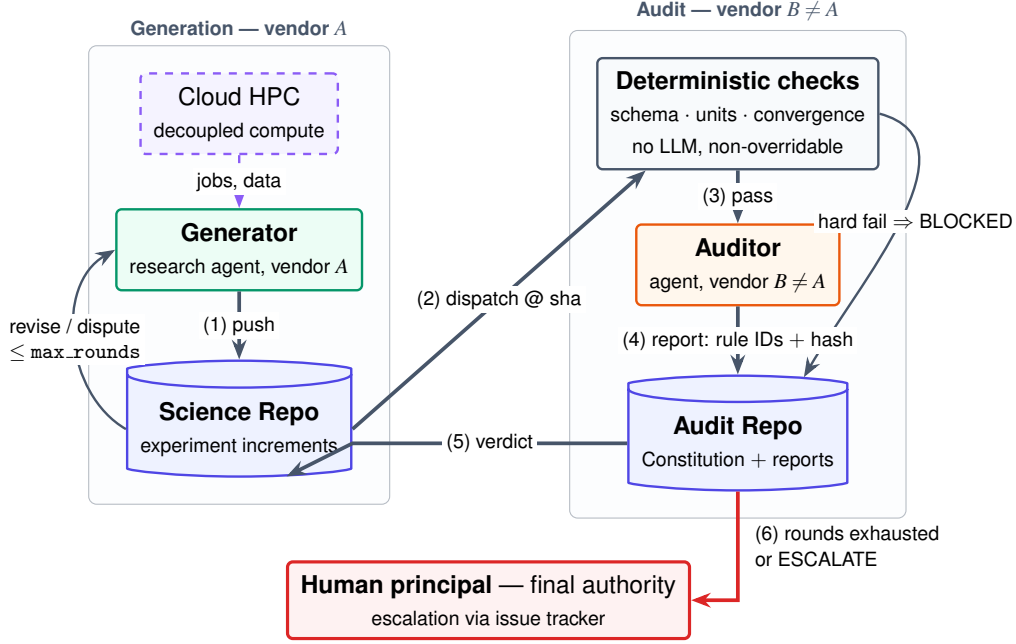


Figure 1: The CrossAudit loop. Solid numbered arrows follow one experiment increment; the deterministic layer runs before any model (I4); the revision loop is bounded (I5); humans are interrupted only by escalation (I6).

Supervision as an artefact. Software engineering solved an analogous trust problem not by making programmers infallible but by making quality mechanisms *inspectable*: version control, continuous integration, review trails. Reproducibility initiatives in science point the same way — the unit of trust is the auditable record, not the author’s competence [10]. CrossAudit transplants this stance to agentic science: the supervision history itself becomes a first-class, versioned, citable research artefact.

3 The CrossAudit protocol

3.1 Objects and roles

CrossAudit supervises a pipeline in which a **Generator** agent (vendor A) produces discrete *experiment increments* — commits under a designated path of a **Science Repository**. Each increment is audited by an **Auditor** agent (vendor $B \neq A$) against a versioned, human-authored rulebook, the **Constitution**, held with all *audit reports* in a separate **Audit Repository**. A **Deterministic Check Layer** (DCL) — plain scripts with no learned component: schema validation, unit and dimension analysis, convergence assertions, provenance completeness — runs before any model reads anything. A **Human Principal** authors the Constitution and receives escalations, but sits deliberately outside the per-increment loop. Compute (in the reference deployment, jobs on cloud HPC) is decoupled from supervision: verdicts gate the *next* increment, never running jobs.

The two-repository split is separation of powers made mechanical: the Auditor holds no write access to the Science Repository, the Generator none to the Audit Repository, so neither agent can alter the other’s record even in error.

3.2 Six invariants

An implementation is CrossAudit exactly when it preserves the following.

I1 — Heterogeneity. The Auditor’s model family differs from the Generator’s. This removes self-preference bias by construction [5] and decorrelates vendor-idiosyncratic failure modes.

- I2 — Ledger completeness.** Every protocol artefact — increment, report, verdict, dispute, escalation, and every change to the Constitution — is a commit (or an issue) in one of the two repositories. No supervision state lives only in model context or platform logs. The history is replayable by a third party from the repositories alone.
- I3 — Citation validity.** An audit report must cite the rule IDs it applied and the commit hash of the Constitution version in force. A report that cites nothing, or an unverifiable version, is *invalid* and treated as an Auditor failure — it escalates; it can never pass an increment.
- I4 — Determinism first.** The DCL runs before any LLM audit. A DCL hard failure yields BLOCKED regardless of any model’s opinion, and no model may waive a DCL verdict. Rationale: heterogeneous models still share the public corpus and hence its errors; the only supervision layer with failure modes fully independent of every model is one with no learned component at all.
- I5 — Bounded revision.** At most `max_rounds` Generator–Auditor exchanges per increment (default three). Exhaustion escalates. This is the oscillation guard: without it, two mutually-correcting agents can disagree indefinitely, each round locally plausible and the pair jointly divergent.
- I6 — Graded interruption.** Findings carry exactly two severities. BLOCKER (objective defect: DCL failure, internal contradiction, missing provenance, method–declaration mismatch) gates the increment. ADVISORY (judgement: parameter taste, style, scope) is recorded for asynchronous human reading and never gates. Humans are interrupted only by ESCALATE. Running compute is never interrupted at all.

Two severities, deliberately. Every additional level invites severity-inflation negotiation between the agents; where a finer distinction is needed it belongs in the rule’s acceptance criterion, not in the escalation lattice.

3.3 The Constitution

The Constitution is the protocol’s quality ceiling: the Auditor cannot exceed the rules it is given, and the Generator will learn whatever the rules fail to say. CrossAudit therefore treats it as law rather than documentation. Every rule carries a stable identifier (CA-DATA-001), a severity, a path scope, a *decidable* acceptance criterion, and a rationale. Decidability is the load-bearing requirement: a criterion a competent reader can evaluate against named artefacts without further judgement. What cannot be phrased decidably is either demoted to ADVISORY or — better — mechanised and moved into the DCL. Meta-rules govern the audit itself: missing evidence is a finding, never an implicit pass (CA-META-001); a report without citations is invalid (CA-META-002, enforcing **I3**); the Auditor may not issue a BLOCKER without citing a BLOCKER-severity rule — it does not legislate taste (CA-META-003); and matters beyond the increment-plus-Constitution evidence base must escalate rather than be guessed (CA-META-004). Constitution changes are commits reviewed by the human principal; the loop can propose rules but never enact them.

3.4 Rounds, disputes, termination

Round k for an increment: the Auditor publishes report R_k with a verdict. On BLOCKED, the Generator either *revises* (a new commit, opening round $k+1$) or *disputes* specific findings by rule ID, with grounds. A disputed finding returns to the Auditor once: it is withdrawn with a logged reason, or upheld. An upheld dispute at the round bound escalates. Disputes exist because the Auditor is fallible in both directions — a false-positive BLOCKER, silently obeyed, degrades the science as surely as a false negative — and the one-shot rule prevents the dispute channel from becoming the oscillation it exists to control. Termination is structural: each increment ends in PASS or in human hands after at most `max_rounds` exchanges.

4 Reference implementation and deployment

4.1 Mechanics

The reference implementation¹ uses no infrastructure beyond GitHub itself. A push under `experiments/` triggers a cross-repository dispatch carrying the commit SHA, changed paths, and round number (read from a commit trailer). The Audit Repository’s workflow checks out the science repository *pinned at the audited SHA*, asserts **I1** from configuration, runs the DCL, then invokes the Auditor via an OpenAI-compatible endpoint with the Constitution, the DCL output, and the increment content as fenced data. The report is committed to the ledger; the verdict is dispatched back; the science repository routes it — proceed, revise, or open an escalation issue. The whole implementation is three workflow files and two Python programs (a check runner and an auditor runner) of a few hundred dependency-light lines; configuration is a single `crossaudit.yml` declaring vendors, `max_rounds`, and escalation policy.

Three implementation details carry protocol weight. First, the auditor runner validates the model’s reply against a fixed schema — verdict, findings with severities and rule citations — and any malformed or citation-free reply is converted to ESCALATE (**I3**); the most common failure shape of an LLM judge, the confident empty report, is thereby made harmless. Second, increment content is passed to the Auditor strictly as data, size-bounded and fenced, with the system prompt instructing that embedded instructions be ignored; the Auditor has no tools. Third, verdict synthesis is code, not model output: DCL hard failures dominate any model verdict (**I4**).

4.2 A live deployment

CrossAudit runs live in the author’s computational-chemistry pipeline (perovskite additive screening). The generation side is a Claude-based research agent that plans calculations, launches them on Alibaba Cloud E-HPC and a GPU worker, and pushes every increment to a private science repository whose audit policy — an epistemic-status ledger separating established, bounded, and provisional claims, together with a standing retraction log — is itself a committed, versioned document. The audit side is orchestrated by a controller that opens a *cycle* against a fixed science commit: the audit request pins a SHA-256 evidence manifest covering every tracked file, and a Codex-CLI auditor (OpenAI lineage, so **I1** holds against the Claude-based generator) writes exactly four artifacts into an immutable per-cycle directory of a separate audit repository — human-readable report, machine-readable result, run metadata, and a per-file hash manifest — on a single long-lived `audit` branch that later cycles may never rewrite. The controller validates each audit commit before recording the cycle as final, and its own state deliberately lives outside both repositories, so neither agent can tamper with the record of its own supervision. Every input and output lives on GitHub; any single component — generator, auditor, controller host, workstation — can fail without loss of state.

Two refinements of the deployed system go beyond the reference implementation and are worth adopting upstream. First, findings are *evidence-typed*: each carries a policy citation (section and rule ID), a deterministic check identifier where applicable, and a copy-paste *reproduce* command, so a human can re-derive any deterministic finding in seconds. Second, blocking is *scope-granular*: a finding blocks named scopes — production-job submission, claim publication — rather than the pipeline wholesale, refining **I6**’s graded-interruption principle. Human authority sits exactly where the protocol places it: production launches require explicit principal approval, recorded in the ledger. The loop is live in the plainest sense: at the time of writing, its most recent cycle — opened and closed within six minutes on the morning of writing — returned a BLOCK decision on deterministic grounds (manifest-hash disagreements and commit-provenance defects), freezing production submission and claim publication until resolved. Audits gate the research record, not the queue; running E-HPC jobs are never interrupted.

Costs are modest by construction. The DCL is free. LLM audit cost scales with the size of the *increment*, not of the computation behind it: auditing a few hundred lines of results and metadata costs cents, while the experiment it certifies may represent hours of HPC time. Supervision priced at roughly

¹<https://github.com/dongzhaohe321418-lab/crossaudit> (MIT licence).

the cost of the documentation it reads is, we suggest, the cheapest layer of assurance computational science has ever been offered.

We deliberately make no quantitative claims about detection rates here. The honest evaluation — seeded-defect recall across heterogeneous versus homogeneous auditor pairings, false-positive burden, escalation precision — requires a defect corpus and is future work (§5); the present paper’s claim is architectural, and the deployment’s role is existence proof: the protocol runs unattended, at single-researcher scale, on commodity infrastructure.

5 Threat model and limitations

We state plainly what CrossAudit withstands and what it does not.

- **Self-preference and vendor-correlated error** are addressed by **I1**. *Corpus*-correlated error is not: where the shared literature is wrong or outdated, both vendors can agree confidently and wrongly. Mitigation is **I4** for everything mechanisable, and the human’s asynchronous reading of ADVISORY trails for what is not. This residual is irreducible by adding more LLM auditors, however many.
- **Prompt injection**. An increment could embed text addressing the Auditor. The Auditor runs toolless against a fixed reply schema with content fenced as data; and because DCL verdicts are non-overridable, injection cannot waive the layer where hard defects are caught. Injection shaping the *LLM-side* report remains partially open, as it does for all current LLM-judge systems [6].
- **Auditor failure** — empty, malformed, or rule-free reports — converts to escalation (**I3**), never to silent passage. The cost is human attention, which is the correct currency for an unreliable auditor.
- **Oscillation and deadlock** are bounded by **I5**; the failure mode degrades to a human decision with the full exchange on the ledger, which is the intended floor.
- **Goodharting the Constitution**. A capable Generator will satisfy the letter of decidable rules. Decidability makes this visible rather than solved: the human reviews rule–outcome drift on the ledger and legislates accordingly. The Constitution is a living document; its growth from escalations is the system learning.
- **Collusion** in the literal sense is excluded by architecture — the agents share no channel except committed artefacts, so “collusion” would have to be conducted in public, on the record. Convergent style over many rounds is the realistic soft risk, met by Constitution review cadence.
- **Scope**. CrossAudit audits the research *record*, not the world: it cannot detect a miscalibrated instrument, a wrong dataset honestly described, or fraud upstream of the repository. It is also currently specified for single-generator pipelines; multi-agent generation raises attribution questions we do not address here.

6 Discussion

Continuous integration for scientific claims. The precise ambition of CrossAudit is the one CI achieved for software: not proof of correctness, but a cheap, always-on, inspectable floor under quality, with human judgement redeployed to where it discriminates — the rules, the escalations, and the science — rather than spent uniformly on every increment. The analogy is exact in one further respect: CI succeeded because it required no institutional permission. A single developer could adopt it unilaterally, and its artefacts (badges, logs) became social proof. CrossAudit is designed for the same adoption path: one researcher, two repositories, an afternoon.

Relation to platform-scale systems. CrossAudit is complementary to, not competitive with, systems such as the AI co-scientist [4] or the AI Scientist [3]. Those systems generate; CrossAudit supervises

whatever generates. Indeed the sharpest version of our proposal is addressed to exactly such platforms: internal critique stages would cost little to re-instantiate across vendor lines and to externalise onto public ledgers, and the resulting audit trails would give publishers and funders — currently being asked to trust process descriptions [11] — something they can actually inspect.

What the ledger buys science socially. A complete, public supervision history changes the character of machine-generated results. A claim arrives not as prose plus reputation but as prose plus its audit record: which rules it was checked against, what was contested, what a differently-trained model conceded or refused. Reviewers can spot-check the ledger instead of re-deriving trust from nothing. Failed audits are data too: a public record of what auto-generated science gets wrong, per rule and per field, is precisely what the community lacks in the current platform era.

The ledger as a data asset. A CrossAudit deployment produces, as a by-product, something the field currently lacks entirely: a corpus of *supervised machine science*. Each (increment, findings, dispute, resolution) tuple is a labelled example of scientific work judged by an independent party under explicit rules — high-grade training signal for better generator and auditor agents alike. Aggregated across deployments, failed audits form a public error taxonomy of agentic research: what machine-generated science actually gets wrong, per rule, per field, over time. Whoever runs the loop accumulates this asset; a community that runs it openly accumulates it as a commons.

Constitutions as community objects. Because the Constitution is a plain, forkable file with stable rule IDs, it can outgrow any single deployment: a research community can maintain a shared domain constitution the way communities maintain style guides and lint rules; a journal could publish the constitution it expects machine-assisted submissions to have been audited under, and accept the ledger link alongside the manuscript, exactly as data-availability statements are accepted today; funders could write “public supervision ledger” into terms. For regulated settings (pharmaceutical and clinical computation among them), the ledger’s attributability and contemporaneous, tamper-evident record are properties audit regimes already demand of human work. And a repository badge — increments audited, escalations raised, constitution version — would do for machine science what CI badges did for software: compress an inspectable process into a glanceable, verifiable signal. None of this requires new infrastructure; it requires only that the artefact exist, which is what the protocol guarantees.

Management by exception, at research scale. As agent labour becomes cheap, the binding constraint of agentic science shifts from compute to human attention. CrossAudit’s economic content is a reduction in supervision cost from $O(\text{increments})$ to $O(\text{escalations})$: the human reads verdicts by exception, advisories asynchronously, and rules on demand. This is the managerial precondition for one researcher credibly directing several agent pipelines at once — and it generalises beyond science to any agentic work product that arrives as discrete increments whose quality is partially rule-expressible, from code to quantitative analysis.

7 Conclusion

CrossAudit reframes supervision of autonomous research from a platform feature into a public, versioned artefact produced by structurally independent reviewers. Its ingredients are deliberately unglamorous — a second vendor, a rulebook with IDs, scripts before models, a bounded loop, and git underneath everything — because unglamorous ingredients are what individual researchers can adopt today without permission. The protocol’s six invariants are, we believe, close to the minimum that buys decorrelated review, replayable history, and human authority at the boundary; everything else in this paper is reference detail, and the repositories are public. An AI scientist should not grade its own homework. With two repositories and two API keys, it no longer has to.

Acknowledgements. The CrossAudit reference implementation and the deployment described in §4 were developed with the assistance of the very class of agentic tools this paper proposes to supervise; the irony is intended.

References

- [1] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv:2408.06292*, 2024.
- [2] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv:2504.08066*, 2025.
- [3] C. Lu et al. Towards end-to-end automation of AI research. *Nature*, 2026. <https://doi.org/10.1038/s41586-026-10265-5>.
- [4] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, et al. Towards an AI co-scientist. *arXiv:2502.18864*, 2025.
- [5] A. Panickssery, S. R. Bowman, and S. Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. *arXiv:2404.13076*.
- [6] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. *arXiv:2306.05685*.
- [7] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.
- [8] N. J. Szymanski, B. Rendy, Y. Fei, R. E. Kumar, et al. An autonomous laboratory for the accelerated synthesis of novel materials. *Nature*, 624:86–91, 2023.
- [9] M. D. Skarlinski, S. Cox, J. M. Laurent, J. D. Braza, et al. Language agents achieve superhuman synthesis of scientific knowledge. *arXiv:2409.13740*, 2024.
- [10] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.
- [11] Nature (news feature). AI scientists are changing research — institutions, funders and publishers must respond. *Nature*, 2026. <https://doi.org/10.1038/d41586-026-00934-w>.